Lab 6 on different dataset

Anshul Katkar

C

C3 batch

32

```
# 1. Import Libraries


import tensorflow as tf
import numpy as np
import os

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.20.0
```

```
# 2. Load Dataset


(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

print("Training data shape:", x_train.shape)
print("Testing data shape :", x_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Training data shape: (60000, 28, 28)
Testing data shape : (10000, 28, 28)
```

```
# 3. Define Class Names


class_names = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot"
]

print("Classes:")
for i, name in enumerate(class_names):
    print(i, ":", name)
```

```
Classes:
0 : T-shirt/top
1 : Trouser
2 : Pullover
```

```
3 : Dress
4 : Coat
5 : Sandal
6 : Shirt
7 : Sneaker
8 : Bag
9 : Ankle boot
```

```
# 4. Normalize Data


x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("Data normalization completed.")
print("Training data range:", x_train.min(), "to", x_train.max())
```

```
Data normalization completed.
Training data range: 0.0 to 1.0
```

```
# 5. Build Neural Network Model


model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

print("Neural network created successfully.")
```

```
Neural network created successfully.
```

```
# 6. Compile Model


model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully.")
```

```
Model compiled successfully.
```

```
# 7. Display Model Architecture


model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)

```
# 8. Train Model
```

```
print("Training TensorFlow model...")

history = model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_split=0.1,
    batch_size=32
)

print("Training completed.")
```

```
Training TensorFlow model...
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.8233 - loss: 0.5104 - val_accuracy: 0.8508 - val_loss: 0.4168
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8633 - loss: 0.3842 - val_accuracy: 0.8668 - val_loss: 0.3596
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8757 - loss: 0.3454 - val_accuracy: 0.8663 - val_loss: 0.3658
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 3ms/step - accuracy: 0.8829 - loss: 0.3203 - val_accuracy: 0.8648 - val_loss: 0.3655
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8885 - loss: 0.3012 - val_accuracy: 0.8743 - val_loss: 0.3529
Training completed.
```

```
# 9. Evaluate TensorFlow Model


loss, accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("=====================================")
print("TENSORFLOW MODEL RESULTS")
print("=====================================")
print("Test Loss     :", loss)
print("Test Accuracy :", accuracy)
```

```
=====================================
TENSORFLOW MODEL RESULTS
=====================================
Test Loss     : 0.3757721781730652
Test Accuracy : 0.8632000088691711
```

```
# 10. Convert TensorFlow Model to TFLite


print("Converting TensorFlow model to TFLite...")

converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

print("TFLite conversion completed successfully.")
```

```
Converting TensorFlow model to TFLite...
Saved artifact at '/tmp/tmpiffvefee'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28), dtype=tf.float32, name='keras_tensor')
Output Type:
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)
Captures:
  136978871579088: TensorSpec(shape=(), dtype=tf.resource, name=None)
  136978871580048: TensorSpec(shape=(), dtype=tf.resource, name=None)
  136978871579664: TensorSpec(shape=(), dtype=tf.resource, name=None)
  136978871580432: TensorSpec(shape=(), dtype=tf.resource, name=None)
TFLite conversion completed successfully.
```

```
# 11. Save TFLite Model
```

```
tflite_filename = "fashion_mnist_model.tflite"

with open(tflite_filename, "wb") as f:
    f.write(tflite_model)

print("TFLite model saved as:", tflite_filename)
```

```
TFLite model saved as: fashion_mnist_model.tflite
```

```
# 12. Calculate TFLite Model Size


size = os.path.getsize(tflite_filename)

print("=====================================")
print("TFLITE MODEL SIZE")
print("=====================================")
print("Model Size in Bytes :", size)
print("Model Size in KB    :", size / 1024)
```

```
=====================================
TFLITE MODEL SIZE
=====================================
Model Size in Bytes : 409708
Model Size in KB    : 400.10546875
```

```
# 13. Load TFLite Model


interpreter = tf.lite.Interpreter(model_path=tflite_filename)
interpreter.allocate_tensors()

print("TFLite model loaded successfully.")
```

```
TFLite model loaded successfully.
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:     Warning: tf.lite.Interpreter
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.

  warnings.warn(_INTERPRETER_DELETION_WARNING)
```

```
# 14. Get Input and Output Details


input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

print("=====================================")
print("INPUT DETAILS")
print("=====================================")
print("Input Index :", input_details[0]["index"])
print("Input Shape :", input_details[0]["shape"])
print("Input Type  :", input_details[0]["dtype"])

print("\n=====================================")
print("OUTPUT DETAILS")
print("=====================================")
print("Output Index :", output_details[0]["index"])
print("Output Shape :", output_details[0]["shape"])
print("Output Type  :", output_details[0]["dtype"])
```

```
=====================================
INPUT DETAILS
=====================================
Input Index : 0
Input Shape : [ 1 28 28]
Input Type  : <class 'numpy.float32'>
```

```
======================================
OUTPUT DETAILS
======================================
Output Index : 14
Output Shape : [ 1 10]
Output Type  : <class 'numpy.float32'>
```

```
# 15. Select Test Sample


sample = x_test[0:1]
actual_label = y_test[0]

print("Actual Label Number:", actual_label)
print("Actual Class       :", class_names[actual_label])
```

```
Actual Label Number: 9
Actual Class       : Ankle boot
```

```
# 16. Perform TFLite Inference


interpreter.set_tensor(
    input_details[0]["index"],
    sample
)

interpreter.invoke()

print("TFLite inference completed.")
```

```
TFLite inference completed.
```

```
# 17. Get Output and Prediction


output = interpreter.get_tensor(
    output_details[0]["index"]
)

prediction = np.argmax(output[0])
predicted_class = class_names[prediction]
confidence = output[0][prediction] * 100

print("======================================")
print("TFLITE INFERENCE RESULT")
print("======================================")
print("Actual Label Number    :", actual_label)
print("Actual Class           :", class_names[actual_label])
print("Predicted Label Number :", prediction)
print("Predicted Class        :", predicted_class)
print("Prediction Confidence  :", confidence, "%")
```

```
======================================
TFLITE INFERENCE RESULT
======================================
Actual Label Number    : 9
Actual Class           : Ankle boot
Predicted Label Number : 9
Predicted Class        : Ankle boot
Prediction Confidence  : 71.87656 %
```

```
# 18. Display All Class Probabilities


print("====================================")
print("CLASS PROBABILITIES")
print("====================================")
```

```
for i in range(10):
    probability = output[0][i] * 100
    print(f"{i} - {class_names[i]:15s} : {probability:.2f}%")
```

```
======================================
CLASS PROBABILITIES
======================================
0 - T-shirt/top     : 0.01%
1 - Trouser         : 0.00%
2 - Pullover        : 0.00%
3 - Dress           : 0.00%
4 - Coat            : 0.00%
5 - Sandal          : 3.33%
6 - Shirt           : 0.01%
7 - Sneaker         : 24.52%
8 - Bag             : 0.26%
9 - Ankle boot      : 71.88%
```

```
# 19. Check Prediction


if prediction == actual_label:
    print("Prediction is CORRECT.")
else:
    print("Prediction is INCORRECT.")
```

```
Prediction is CORRECT.
```

```
# 20. Test Multiple Samples


correct = 0
total_samples = 10

print("======================================")
print("MULTIPLE SAMPLE TESTING")
print("======================================")

for i in range(total_samples):
    sample = x_test[i:i+1]

    interpreter.set_tensor(
        input_details[0]["index"],
        sample
    )

    interpreter.invoke()

    output = interpreter.get_tensor(
        output_details[0]["index"]
    )

    prediction = np.argmax(output[0])
    actual = y_test[i]

    print(
        "Sample:", i + 1,
        "| Actual:", class_names[actual],
        "| Predicted:", class_names[prediction]
    )

    if prediction == actual:
        correct += 1
```

```
======================================
MULTIPLE SAMPLE TESTING
======================================
Sample: 1 | Actual: Ankle boot | Predicted: Ankle boot
Sample: 2 | Actual: Pullover | Predicted: Pullover
Sample: 3 | Actual: Trouser | Predicted: Trouser
Sample: 4 | Actual: Trouser | Predicted: Trouser
Sample: 5 | Actual: Shirt | Predicted: Shirt
```

```
Sample: 6 | Actual: Trouser | Predicted: Trouser
Sample: 7 | Actual: Coat | Predicted: Coat
Sample: 8 | Actual: Shirt | Predicted: Shirt
Sample: 9 | Actual: Sandal | Predicted: Sandal
Sample: 10 | Actual: Sneaker | Predicted: Sneaker
```

```
# 21. Calculate Multiple Sample Accuracy


sample_accuracy = (correct / total_samples) * 100

print("======================================")
print("MULTIPLE SAMPLE ACCURACY")
print("======================================")
print("Correct Predictions :", correct)
print("Total Samples       :", total_samples)
print("Accuracy            :", sample_accuracy, "%")
```

```
======================================
MULTIPLE SAMPLE ACCURACY
======================================
Correct Predictions : 10
Total Samples       : 10
Accuracy            : 100.0 %
```

```
# 22. Display Final Summary


print("======================================")
print("FINAL SUMMARY")
print("======================================")

print("Dataset              : Fashion-MNIST")
print("Training Samples     :", len(x_train))
print("Testing Samples      :", len(x_test))
print("Image Size           : 28 x 28")
print("Number of Classes    :", 10)
print("TensorFlow Accuracy  :", accuracy)
print("TFLite Model Size    :", size / 1024, "KB")
print("TFLite Model File    :", tflite_filename)

print("\nTensorFlow to TFLite conversion")
print("and Edge AI inference completed successfully.")
```

```
======================================
FINAL SUMMARY
======================================
Dataset              : Fashion-MNIST
Training Samples     : 60000
Testing Samples      : 10000
Image Size           : 28 x 28
Number of Classes    : 10
TensorFlow Accuracy  : 0.8632000088691711
TFLite Model Size    : 400.10546875 KB
TFLite Model File    : fashion_mnist_model.tflite

TensorFlow to TFLite conversion
and Edge AI inference completed successfully.
```

```
# 23. Display a Test Image and Prediction


import matplotlib.pyplot as plt

index = 0

plt.figure(figsize=(4, 4))
plt.imshow(x_test[index], cmap="gray")
plt.title(
    f"Actual: {class_names[y_test[index]]}\n"
    f"Predicted: {class_names[prediction]}"
)
plt.axis("off")
```

```
plt.show()
```

